

Music Playlist Management System

Implementation of a Singly Linked List in an Interactive Web Application

Student Name: Sakshi

Course: DSA / Industrial Practice Lab

GitHub Repository: github.com/sakxhy/music-playlist-dsa

Live Web Demo: sakxhy.github.io/music-playlist-dsa/simple.html

1. Project Overview

The **Music Playlist Manager** is a modern interactive web application developed to demonstrate how fundamental data structures solve real-world problems. In commercial audio streaming applications such as Spotify or Apple Music, users interact with a dynamic song queue—adding songs to play next, appending songs to the end of the queue, skipping to the next track, or reversing the playback order.

This project models this playback system using a **Singly Linked List**. Because audio tracks in a playlist are played sequentially one after another, pointer-based node linking provides an intuitive, elegant, and realistic software implementation.

2. How Singly Linked List is Implemented

A Singly Linked List is a linear data structure composed of individual, dynamically allocated **Nodes**. Unlike contiguous memory arrays, each node in a linked list exists independently and connects to subsequent items via a reference pointer.

2.1 The Song Node (`SongNode`)

Each audio track is encapsulated as an instance of `SongNode`. Every node contains two distinct components:

- **Data Fields:** Metadata properties describing the song, including `title`, `artist`, and `duration`.
- **Pointer Field (`next`):** A pointer variable storing the memory reference of the subsequent `SongNode`. If the node is the last song in the playlist, `next` is assigned `null`.

```

+-----+
|                               |
|                               |
+-----+
| Data:  title, artist, duration |
| Pointer: next —> (Points to next SongNode, or NULL) |
+-----+

```

```

class SongNode {
    constructor(title, artist, duration) {
        this.title = title;           // Song Name
        this.artist = artist;         // Artist Name
        this.duration = duration;     // Song Length (mm:ss)
        this.next = null;             // Pointer link to next track
    }
}

```

2.2 The Playlist Manager Structure

The higher-level playlist operations are managed by a `PlaylistManager` class. This controller tracks three essential reference markers:

- **head** : Points directly to the first node of the playlist. If `head === null`, the playlist is empty.
- **currentTrack** : Points to the specific node currently being played.
- **size** : An integer counter tracking the total number of songs in the queue.

```

HEAD                                     TAIL
[Song 1] ——.next—> [Song 2] ——.next—> [Song 3] ——.next—> NULL
(Active)           (Next Up)           (Last Song)

```

```

class PlaylistManager {
    constructor() {
        this.head = null;           // Entry point to linked list
        this.currentTrack = null;   // Active song pointer
        this.size = 0;              // Total song count
    }
}

```

3. Basic Operations & Implementation Details

3.1 Insertion Operations

A. Insertion at Head ("Play Next")

Inserts a newly added song at the very beginning of the playlist so that it is positioned as the immediate next song to play.

1. Create a new node `newNode = new SongNode(title, artist, duration)` .
2. Link the new node forward: `newNode.next = this.head` .
3. Reassign the head reference: `this.head = newNode` .
4. Increment the playlist size counter.

```
insertAtHead(title, artist, duration) {
    const newNode = new SongNode(title, artist, duration);
    newNode.next = this.head;
    this.head = newNode;
    this.size++;

    if (!this.currentTrack) {
        this.currentTrack = this.head;
    }
}
```

B. Insertion at Tail ("Add to Queue")

Appends a new track to the very end of the playlist.

1. Allocate the new `SongNode` .
2. If the playlist is empty (`this.head === null`), set `this.head = newNode` .
3. Otherwise, initialize a temporary traversal pointer `temp = this.head` .
4. Advance `temp = temp.next` until reaching the last node where `temp.next === null` .
5. Connect the last node to the new node: `temp.next = newNode` .

```
insertAtTail(title, artist, duration) {
    const newNode = new SongNode(title, artist, duration);
    if (!this.head) {
        this.head = newNode;
        this.currentTrack = newNode;
    } else {
        let temp = this.head;
        while (temp.next !== null) {
            temp = temp.next;
        }
        temp.next = newNode;
    }
    this.size++;
}
```

C. Insertion at a Specific Position

Places a song at a designated 0-based index. Traverses to index `k - 1`, connects `newNode.next = prev.next`, and sets `prev.next = newNode`.

3.2 Deletion Operations

A. Deletion from Head

Removes the first track from the playlist by advancing the head reference: `this.head = this.head.next`.

```
deleteHead() {  
    if (!this.head) return false;  
    const deletedNode = this.head;  
    this.head = this.head.next; // Move head forward  
    if (this.currentTrack === deletedNode) {  
        this.currentTrack = this.head;  
    }  
    this.size--;  
    return true;  
}
```

B. Deletion by Position / Specific Song

Removes any song node from the middle or end of the playlist without breaking chain continuity.

1. If index is 0, invoke `deleteHead()`.
2. Maintain two stepping pointers: `prev` and `curr`.
3. Traverse until `curr` reaches the target song to be deleted.
4. Bypass the target node: `prev.next = curr.next`.
5. Decrement the playlist size.

Before Deletion:

[Song A] —.next—> [Song B (Target)] —.next—> [Song C]

After Deletion (Bypassing Song B):

[Song A] —.next—> [Song C]

```

deleteAtPosition(index) {
  if (!this.head || index < 0 || index >= this.size) return false;
  if (index === 0) return this.deleteHead();

  let prev = null;
  let curr = this.head;
  for (let i = 0; i < index; i++) {
    prev = curr;
    curr = curr.next;
  }

  prev.next = curr.next; // Bridge pointer over deleted node
  if (this.currentTrack === curr) {
    this.currentTrack = curr.next || this.head;
  }
  this.size--;
  return true;
}

```

3.3 Traversal & Playback Operations

A. Sequential Playback ("Next Song")

Advances playback to the subsequent track in the chain. Reads `this.currentTrack.next` and reassigns `this.currentTrack`. If `.next` is `null`, playback concludes cleanly.

```

playNext() {
  if (this.currentTrack && this.currentTrack.next) {
    this.currentTrack = this.currentTrack.next;
    return true;
  }
  return false; // Reached end of playlist (NULL)
}

```

B. Visual Rendering Traversal

Traverses from `this.head` node-by-node, rendering each song's title, artist, duration, and an explicit visual pointer arrow (— .next —>) on the webpage. Upon encountering `null`, it renders a terminal `[NULL (End of Chain)]` box.

C. Searching a Track

Iterates sequentially starting from `head`, checking if the query string matches any node's `title` or `artist`. When found, the node is highlighted and scrolled into focus.

3.4 In-place Playlist Reversal

Inverts the playlist order so that the last track becomes the new `head`. It reverses pointer orientations in a single linear pass using three pointers (`prev` , `curr` , `next`):

```
Initial Order:  [Song 1] —> [Song 2] —> [Song 3] —> NULL
Reversed Order: [Song 3] —> [Song 2] —> [Song 1] —> NULL
```

```
reverse() {
  let prev = null;
  let curr = this.head;
  let next = null;

  while (curr !== null) {
    next = curr.next; // 1. Save subsequent node reference
    curr.next = prev; // 2. Reverse pointer to previous node
    prev = curr;      // 3. Move prev forward
    curr = next;      // 4. Move curr forward
  }
  this.head = prev;  // 5. Update head to former tail
}
```

3.5 Repeat Mode / Loop Handling

When repeat mode is active, the tail's `.next` connects back to `head`, forming a circular sequence. To ensure safety and prevent browser crashes during DOM rendering, the application executes **Floyd's Cycle-Finding Algorithm** (using dual `slow` and `fast` pointers) to detect loops dynamically.

4. Interactive User Interface Features

- **Simple Visualizer View (`simple.html`):** Beginner-friendly interface showcasing nodes with `HEAD` , `PLAYING` badges, visual pointer arrows, and 1-click test presets (Pop Hits, Rock Classics, Coding Chill).
- **Studio View (`index.html`):** Full-featured audio player simulator with sound wave canvas animations, node search, and position-based insertions.

Live Cloud Deployment: The project is published and permanently accessible via GitHub Pages at:
<https://sakxhy.github.io/music-playlist-dsa/simple.html>

5. Conclusion

This project provides a complete, working implementation of a **Singly Linked List** applied to a real-world multimedia playback system. By mapping physical songs to nodes and playback progression to pointer traversals, it clearly demonstrates dynamic node creation, insertion at head/tail, deletion by index, linear traversal, and in-place list reversal.
